Kelal Studio — Product Requirements Document

Synthesized from the Program Brief and the Verified Technical Gap Analysis

Version 1.0 · Prepared 12 August 2026 · Technical facts verified 11 August 2026, revalidate by 11 September 2026

Contents

How to read this PRD

Every requirement below is one of three things, and it is labeled where the distinction matters:

- **Decided** — carried directly from the Program Brief, or a judgment call this PRD makes explicitly, with reasoning, so it can be overturned in writing rather than silently.
- **Recommended** — an engineering judgment from the technical gap analysis. Treated as a working requirement here, but the reasoning is preserved so it can be revisited.
- **Open** — genuinely unresolved. These are **not** requirements. They live in the Open Questions Register (Section 14) with an owner and must not be treated as decided by anyone building against this document, including an AI coding tool. If this document is used to drive AI-assisted implementation, the coding tool must be instructed explicitly: **do not resolve an Open item by picking the most plausible option — implement around it, flag it, and stop.**

This document targets AI-assisted implementation directly: reasoning is kept inline next to each decision (a conclusion without its reasoning is one an AI tool will confidently override), quantities are shown as arithmetic rather than bare results (Section 12), and each feature section states its failure path explicitly, because generated implementations reliably build the success path and omit the failure path otherwise.

1. Product Overview

Kelal Studio is an automated social-media content engine for Ethiopian businesses, marketing agencies, and content creators. It is **not** a manual design tool. Its value is collapsing a multi-step, multi-language content workflow — draft, translate, typeset, design, export — into a single pipeline:

A user types a short business update in English or Amharic (e.g. “Caramel latte discount this afternoon, playful tone”). The system generates a bilingual caption with localized Ethiopian hashtags, composes a branded graphic (and optionally a short video teaser) with correct Ethiopic typography, and hands the user a ready-to-publish asset via the phone’s native Share Sheet — with the caption copied to the clipboard for manual paste into Instagram, Telegram, or TikTok.

The product ships as a Flutter mobile app (iOS + Android, primary composer surface) and a React web portal (brand configuration and admin, per the platform split in Section 4), backed by a Go API engine. It launches as a **supervised public beta**: “beta” means resource limits and user caps, not reduced quality. By Week 4, real users on Ethiopian mobile networks must be able to sign up, generate content, and publish without technical assistance.

1.1 System Architecture

The architecture has one structural rule that shapes everything else in this document: **no feature code talks to an AI provider directly.** Every generation call passes through the Provider Abstraction Layer (Section 10.1), which is what makes per-user quota enforcement, response caching, and provider failover possible to implement at all, and what contains the blast radius when a free-tier vendor changes terms or goes down without notice — which Section 12’s evidence shows is a likely event within the project’s horizon, not a hypothetical one.

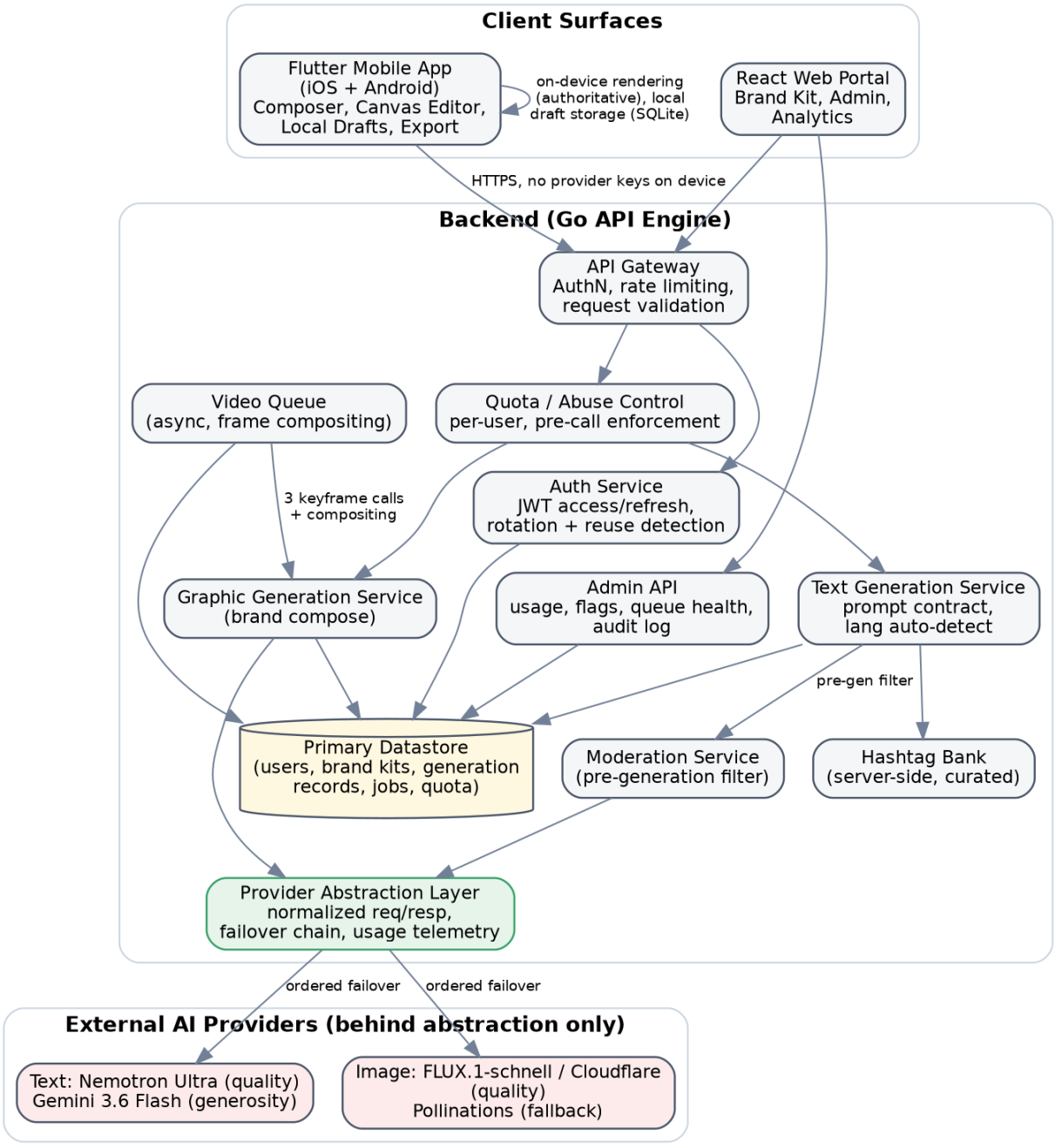

Figure 1: System Architecture — client surfaces, backend services, and external AI providers behind a single abstraction layer

2. Goals

2.1 Launch goal

Ship a production-ready, supervised public beta by the end of Week 4, meeting the Working Product Standard (Section 9.1) end to end, verified on physical hardware under simulated 3G/4G conditions.

2.2 What "success" is measured against

Delivery is weighted evenly across five dimensions, each 20%: architectural quality and reliable failure handling; local product judgment (evidence-based scope choices, genuinely usable Amharic support); operational rigor under spotty networks; team process adherence; and documentation/communication quality.

2.3 Numeric success metrics — Open

Metrics to collect are named (generation counts, rendering latency, export completion rate, error rate, user engagement) but no target is set for any of them, and "activated user" is undefined. **The product team must set a numeric target for each named metric before launch** (Section 14); without a target, launch telemetry has nothing to pass or fail against.

---

3. Target Users

Primary audience: Ethiopian business owners (coffee shops, restaurants, retail, salons, electronics, transport, tutoring, etc.), marketing agencies, and independent content creators who currently manage social content manually across Telegram, Instagram, and TikTok.

Platform-specific audiences:

- **Mobile app:** on-the-go business owners and creators — entering an idea and exporting content in the field.
- **Web portal:** agency managers and desktop admins — configuring brand kits and monitoring usage across accounts.

Personas — not yet developed. These three audiences are named but not built into personas with goals, device constraints, connectivity assumptions, or language/literacy profiles; "Persona & Workflow Insights Analysis" is a listed final deliverable, confirming this research does not yet exist. Two specific assumptions about this audience are **unverified and must be resolved by user research before dependent screens are designed**:

- Whether target users can reliably type Amharic on their phones, or primarily use transliterated/Latin input — affects Idea Composer design (OQ-15).
- Whether the Ethiopian calendar and time conventions should be reflected in the UI, particularly for Local Post Reminders (OQ-16).

---

4. Platform Surface Split

Mobile Application (iOS & Android)	Web Management Portal
Idea entry with language toggle	Brand Kit configuration & logo management
Interactive Ethiopic canvas editor	Admin controls & queue monitoring
Offline draft management	Usage analytics & telemetry

Mobile Application (iOS & Android)	Web Management Portal
Native OS Share Sheet export	Master asset history views

Under the pre-approved descope ladder (Section 5.6), if Week 3 falls behind, the web portal collapses to Admin + Brand Kit only, and the mobile app becomes the sole composer.

5. Core Features & Scope

5.1 Four operational layers

1. **Localized Context & Copywriting Engine** — automated EN/Amharic caption generation and local hashtag curation.
2. **Brand Identity & Typography Guardrails** — brand presets, color palette enforcement, Noto Sans Ethiopic overlays.
3. **Multi-Format Asset Generation Pipeline** — bilingual captions, key visuals, async video teasers.
4. **Native Distribution Loop** — local draft storage, OS Share Sheet integration, clipboard handoff.

5.2 P0 — Non-negotiable for launch

- User Account Management (email/password registration, login, password reset) **+ account deletion** (added — see 5.5)
- Idea Composer (free-text input, English/Amharic, platform switcher)
- Bilingual Caption & Hashtag Generator
- Automated Content Safety Filter (pre-generation)
- Local Draft Persistence (offline-capable, on-device)
- Brand Kit Setup (logo, colors, brand name, tone, contact info)
- Branded Visual Generator (1:1 and 4:5 graphics with palette + logo)
- Touch Canvas Text Overlay (1–2 editable text layers, Ethiopic layout rules, safe zones)
- One-Tap OS Export (image save + caption-to-clipboard + Share Sheet)
- Short Video Teasers (background queue, 15–30s, from storyboards)
- Local Post Reminders (scheduled notifications into the export flow)
- Admin Management Portal (user limits, usage analytics, flagged-prompt audit log)

5.3 P1 — Built only after P0 passes the Working Product Standard

- One-Tap Style Variations (“more casual,” “formal,” “translate”)
- Cloud Asset Library (backup + searchable index of past output)
- Multi-Brand Switching (agencies managing multiple businesses)

5.4 P2 — Explicitly out of scope for V1 (do not build)

- Direct Meta/TikTok API auto-posting (Share Sheet only)
- Manual keyframe timeline video editor
- Multi-user team review/approval workflows
- In-app payment processing

5.5 Addition to P0: Account Deletion — Decided

Not present in the Program Brief’s P0 list or delivery checklist. Apple’s App Store review guidelines require that any app supporting account creation also offer in-app account deletion; the same direction is implied by data-protection best practice regardless of how that verification resolves (OQ-18). **Account deletion is added to P0.** The cost of building it is small; discovering the requirement during Week 4 store review, with no schedule left, is not.

5.6 Pre-approved scope-adjustment sequence (unchanged)

If Week 3 falls behind, descope in this order without delaying launch: 1. Async video render → static image carousel fallback. 2. Style variations → manual caption editing in the draft

screen. 3. Web composer → web portal becomes Admin + Brand Kit only. 4. Multi-frame video → single 10-second static-graphic teaser with audio and animated text.

6. Functional Requirements

6.1 Authentication & Accounts

- Email/password registration, login, and password-reset-via-email.
- JWT access + refresh tokens. **Decided:** refresh tokens rotate on use; reuse of an already-consumed refresh token is detected and treated as a compromise signal.
- Tokens stored in platform secure storage on-device (not plain preferences).
- Password hashing via a modern adaptive algorithm (bcrypt/argon2 — exact parameters an implementation detail, not open).
- Password-reset endpoint returns an identical response whether or not the email exists (prevents account enumeration). Reset tokens are single-use, short-lived, invalidated on use or on password change via another route.
- Account lockout policy on repeated failed logins, balanced against self-inflicted denial-of-service risk.
- Email verification gates content generation (unverified accounts are the cheapest vector for quota abuse).
- **Account deletion**, in-app, per 5.5.

Failure path: wrong password → generic "invalid credentials" (never reveal which field was wrong); expired/reused refresh token → force re-authentication, do not silently issue a new token pair without the reuse-detection check firing.

6.2 Idea Composer & Text Generation

- Free-text input, English or Amharic, with automatic language detection.
- Output: structured bilingual payload — English caption, Amharic translation, call to action, 5–8 localized hashtags.
- Platform switcher formats captions for Instagram, TikTok, and Telegram conventions.
- Speed target: 3 seconds (P50/P95 clarification — **Open**, Section 8.1).
- Fallback path: on LLM timeout/failure, return a pre-cached template response rather than a raw error.
- **Decided:** all provider calls go through the internal provider-abstraction layer (Section 10.1) with an ordered failover chain — no feature code talks to a model vendor directly.

AI model candidates (verified 11 August 2026 — recheck before Week 1 build, pricing/quota change without notice):

	Quality candidate	Generosity candidate
Text	Nemotron Ultra (free, via OpenRouter) — 1M context; 20 req/min, 50 req/day (1,000/day after a one-time $10 credit purchase), enforced globally per account	Gemini 3.6 Flash (Google AI Studio) — free input/output tokens; published rate limits unavailable, must be read from account dashboard
Image	FLUX.1-schnell via Cloudflare Workers AI — 10,000 neurons/day free on every plan, predictable metered overage	Pollinations (no key/account required) — verified working, no published rate limit or documented behavior at limit

	Quality candidate	Generosity candidate
Video	*No candidate exists.* No free text-to-video API exists anywhere (verified against OpenRouter and Google's pricing pages). Video is frame compositing, not generation — see 6.5.	

AI model selection is Open and blocking (OQ-20). Neither source document names a vendor. **Amharic output quality has no defined bar and must be measured before model selection is finalized — Week 1, blocking.** Required before launch:

- A frozen evaluation set of ≥50 representative business inputs (coffee, restaurant, retail, salon, electronics, transport, tutoring; half Amharic, half English; native-speaker authored).
- A scoring rubric: grammatical correctness, register appropriateness, factual faithfulness to input, absence of hallucinated claims (e.g. invented prices), naturalness of translation (not literal).
- A numeric pass threshold a model must clear before selection.
- Native-speaker blind review as the scoring method (automated translation metrics reward literal translation, which is the exact failure mode to avoid).

This evaluation gates the Week 1 model decision — it determines the provider abstraction, cost model, latency profile, and moderation approach, and there is no schedule left to redo it if quality is found inadequate in Week 4.

6.3 Hashtag Curation

- 5–8 localized hashtags per generation (e.g. #AddisAbaba, #EthiopianCoffee, #AddisEats).
- **Decided:** hashtags are selected from a curated, server-side bank seeded from observed real usage on Ethiopian business accounts — not freely generated by the model — plus context-derived tags matched against that bank. Free generation is the likeliest place for a model to invent confident-sounding but nonexistent local references, and a wrong hashtag (unlike a wrong caption) is invisible to a user who cannot easily check it. Bank is server-updatable without an app release. **Open:** ongoing curation ownership and refresh cadence.

6.4 Content Safety / Moderation

- Pre-generation filter screens prompts for inappropriate content or unsupported claims; flagged prompts are logged to the admin audit log, never silently dropped.
- **Open, treated as blocking for public beta (OQ-07):** available moderation classifiers are trained predominantly on English; recall on Amharic is unverified and must be assumed substantially worse. A filter with high English recall and low Amharic recall is more dangerous than no filter — it manufactures false confidence. Requires a measured recall figure against an Amharic adversarial test set before launch.
- **Decided:** moderate the translated English text *in addition to* the original input, not instead of it (one extra inexpensive call; partial mitigation only, must not be described as solving the Amharic-recall gap).
- Required, not yet specified: who reviews the audit log and how often; user-facing refusal message *in the user's input language*; an appeal path (false positives on a business tool block legitimate revenue-generating work).

Failure path: flagged prompt → user sees plain-language refusal in their input language + appeal link; never a raw classifier error code.

6.5 Graphic & Video Generation

- Graphics: brand-composed visuals combining generated base imagery, logo, accent colors, and Ethiopic text overlays.
- Speed target: 6 seconds.
- Aspect ratios: **Open, contradictory in the Program Brief itself (OQ-02).** The brief's feature list and its own composition spec state two ratios (1:1, 4:5); its Layer 3 description and Week 2 task list state three (1:1, 4:5, 9:16). This directly determines render/quota cost and **must be resolved to one canonical list by the product team before the render pipeline is built** — not resolved here.
- Safe zones: top 10% / bottom 15% obstruction avoidance, as stated in the brief. **Open (OQ-06):** these are flat percentages with no reference frame; Instagram Story, TikTok, and a plain feed post each have different interface furniture. A safe-zone table per output ratio *and* per target platform is required, each figure sourced or explicitly tagged as an assumption pending validation (post real test content, screenshot the result — ~1 hour of work).

Video is frame compositing, not generation — Decided.

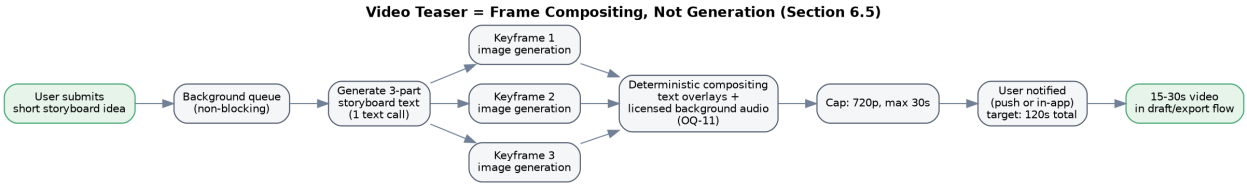

Figure 2: Video teaser pipeline — three image generations plus deterministic compositing, never a video model

The pipeline description ("generate 3-part storyboard text, create keyframe graphics, combine frames with text overlays and background audio") is three image generations plus deterministic compositing; it never actually requires a video model, and none exists free at any price. Cap: 720p, max 30 seconds. This must be communicated to stakeholders as "video teasers delivered via keyframe compositing as specified," not as a missing feature — the real constraint it introduces is image-generation quota (3 extra image calls per video post).

- **Open (OQ-11):** source and license of background music for video teasers. Unlicensed audio in distributed commercial marketing material is a real legal exposure and platforms audio-match against it; requires a licensed or public-domain source before launch.
- Background queue: video generation runs asynchronously, does not block the UI, notifies the user on completion (channel — push or in-app — **Open**; push requires its own platform setup lead time).

6.6 Renderer Architecture — Recommended, adopted

The Program Brief specifies bundling Noto Sans Ethiopic into *both* the mobile app and a server-side rendering pipeline, implying two independent text-layout implementations (client canvas + server graphic generation) that will inevitably diverge (different shaping stacks, line-height defaults, hinting) — surfacing late as "preview doesn't match export," which is the brief's own highest-rated risk (Ethiopic Text Line Collision).

Decision: client-side rendering on-device is the single authoritative renderer. What the user sees in the canvas is the literal artifact exported. Reasoning: eliminates the pre-

view/export mismatch class by construction; satisfies the offline requirement (an offline user with no server round trip can still export); removes server render latency from the 6-second graphic budget; avoids uploading logos/images back and forth on metered mobile data.

Cost of this choice, stated plainly: the web portal cannot independently preview identical output without a second implementation (acceptable — web is admin/brand-kit only), and per-device font rendering can vary across Android versions (mitigated by bundling the font rather than relying on system fonts, which is required regardless). **Open (OQ-03, OQ-04):** formal ratification by the PRD team, and whether Ethiopic genuinely needs no complex text-shaping support in the chosen layout engine (should be confirmed empirically in Week 1, not assumed).

6.7 Ethiopic Typography

- Bundle Noto Sans Ethiopic (SIL Open Font License 1.1 — embedding/bundling in commercial software expressly permitted; font may not be sold standalone; license text must ship with the product in a third-party-license screen).
- Line-height multiplier: start at 1.5–1.6 (Latin defaults near 1.2 are the collision source); validate against a test corpus and fix the final number as a constant.
- Minimum rendered size: measured empirically at export resolution, enforced as a floor (below it, Ethiopic's distinguishing marks become illegible).
- Line-break opportunities: space, plus U+1361 (word separator) and U+1362 (full stop); never break mid-syllable.
- Font fallback: explicit and bundled, never system-font-dependent (a missing glyph renders as an empty box, explicitly prohibited).
- Mixed-script runs (Amharic + Latin brand names + digits + hashtags in one caption): single font family covering all required ranges, to avoid visible baseline/weight mismatch mid-line.
- **Decided: golden-image regression testing.** Build a fixed corpus of Amharic strings (short, long, mixed-script, punctuation-heavy, worst-case longest word) rendered at every supported output size; store approved renders as reference images; run pixel comparison in CI on every commit, failing the build on deviation beyond a small threshold. Converts the brief's manual "verified visually against a checklist" bar into an automated gate. Setup cost: under a day.
- Subsetting: subset non-Ethiopic ranges aggressively for app-size reasons; retain the full Ethiopic range.

6.8 Brand Kit

- Data model: logo file reference, primary/secondary color hex values, brand name, tone-of-voice descriptor, default contact info.
- Backend CRUD with logo upload validation (file type, size limits, aspect-ratio warning).
- Configured on web portal; pulled automatically into mobile generation on next session without manual re-entry.
- File-upload hardening (logo upload is the highest-risk untrusted-input surface in the product): validate by content inspection, not extension or client-supplied MIME type; explicit max dimensions/file size; re-encode every accepted image (neutralizes most embedded-payload attacks as a side effect); strip metadata (security *and* privacy — image metadata can carry location); store outside the web root, serve from a non-executable path; enforce a minimum dimension (a too-small logo looks broken once composited).

6.9 Canvas Editor

- 1–2 editable text layers over the generated graphic.
- Visible safe-zone guides during editing.
- Ethiopic layout rules (line height, padding) applied live in the editor, not only at final render.
- Touch gestures: drag to reposition, pinch/handle-drag to resize, tap to edit text inline.
- **Recommended, adopted (device memory):** edit against a downscaled proxy image; apply the transform to the full-resolution asset only at export. Bounds memory use during the interactive phase on low-RAM Android devices (holding several full-resolution bitmaps risks a crash, not just slowness) and improves gesture responsiveness. A minimum supported device must be named so this is tested, not assumed (Section 7.5).

6.10 Local Draft Persistence

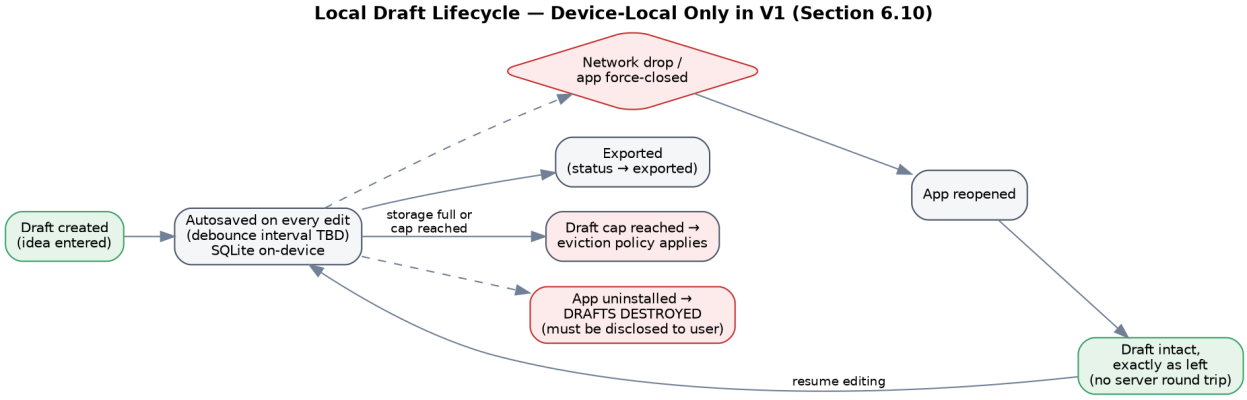

Figure 3: Local draft lifecycle — device-local only in V1, no server sync

- On-device storage (SQLite/Room-equivalent via the Flutter storage layer); autosave on every edit; drafts survive app restart and network loss (validated by an explicit airplane-mode kill/reopen test).
- **Decided: drafts are device-local only for V1 — no server sync.** The brief's own P1 tier already places Cloud Asset Library and multi-device/multi-brand workflows outside V1; with no concurrent-edit surface in V1, there is no conflict to resolve, and building sync machinery for a problem the V1 feature set cannot produce is expensive, hard-to-debug work the schedule doesn't have room for.
- Still required with local-only drafts: storage schema and its migration path; autosave debounce interval; a cap on stored drafts and eviction policy at the cap; behavior when device storage is full (common on low-end devices); explicit disclosure to the user that uninstalling the app destroys drafts.
- **Open (OQ-05):** if the product team overrides local-only and syncs anyway, the conflict rule (e.g. last-write-wins by server timestamp) and its data-loss implications must be stated explicitly and surfaced in the UI — this PRD does not make that call.

6.11 Native Distribution / Export

- One-tap export: save image to phone gallery → copy caption to clipboard → trigger native OS Share Sheet (Instagram, TikTok, Telegram as targets).
- **Verified constraint (not a design flaw in the brief): Instagram does not support prefilled captions from third-party mobile sharing** — deliberate platform policy, not an engineerable-around limitation. Clipboard-copy-then-manual-paste is the correct and

only workaround; the brief's design is right, but it undercounts that this introduces a manual step into a flow billed as frictionless.
- **Decided:** the flow relies on the user long-pressing "Paste" inside the target app themselves. Kelal must never read the clipboard programmatically and must never attempt a programmatic paste — on iOS 16+, an app reading the clipboard triggers an "Allow Paste" prompt, but the prompt does not appear when the user performs the paste gesture themselves (the system treats explicit user action as consent). This is simultaneously the optimal and simplest path.
- Required, not yet specified: a first-run instructional moment teaching the paste step; exact export copy ("Caption copied. Paste it into Instagram after selecting your photo."); whether the "under two minutes" claim includes or excludes the manual paste step; fallback behavior when the target app isn't installed, clipboard permission is denied, or gallery write fails — each needs a plain-language, non-technical message.

6.12 Local Post Reminders

- Scheduled notifications opening a draft directly into the export flow.
- **Open (OQ-16):** whether dates/times should reflect the Ethiopian calendar and time-reckoning convention rather than Gregorian/24-hour — a user-research question, since it's visible on a P0 feature. Regardless of display decision, all timestamps are stored in UTC and converted only at the presentation layer (standard practice, stated explicitly here because reminder features are where it's most often violated).

6.13 Admin Management Portal

- User limits, usage analytics, flagged-prompt audit log, background-queue health monitoring.
- **Decided (required, not stated as such in either source but implied by 6.4 and 7.8):** separate admin authentication requirement, a role model, and an audit trail of administrator actions — an audit log administrators can silently modify is not an audit log.

6.14 Quota & Abuse Control

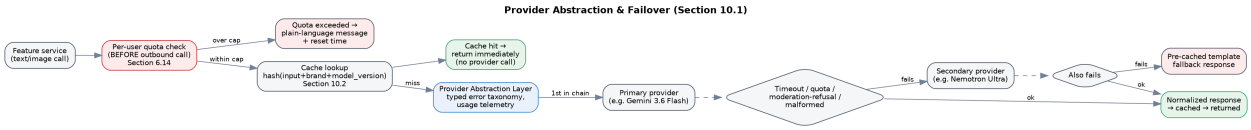

Figure 4: Provider abstraction, quota enforcement, caching, and failover on a single generation call

- **Decided, architecturally required, not present in either source:** per-user quota must be enforced *inside* Kelal, before the outbound provider call — not by observing provider errors after the fact. Provider rate limits are enforced globally per account, not per end user; one enthusiastic user can exhaust the entire beta population's shared daily budget in minutes, and by the time the provider returns an error the shared budget is already gone.
- Rate limiting per user and per IP at the gateway (already in Week 1 plan — keep it there).
- A global daily spend/quota ceiling with automatic shutoff, so a bug or attack can't consume the full budget overnight (not present in either source).
- Registration-abuse controls, since free per-account quota creates an incentive to farm accounts.

- Behavior at quota exhaustion is a **product** decision, not an edge case — under the capacity arithmetic (Section 12), exhaustion is an expected daily operating condition during beta. The product team must decide: does the user see a clear message with a reset time; is remaining quota visible *before* generation is attempted rather than only on refusal; do administrators get alerted before the ceiling is hit rather than after users do.

7. Non-Functional Requirements

7.1 Performance

Area	Brief states	This PRD adds
Text generation	3s	P50 or P95, and under what network condition, is **Open**. A P50-only target is close to meaningless for user-perceived reliability; both should be specified and measured on a real Ethiopian mobile network, not simulated conditions.
Graphics	6s	Same P50/P95 clarification required.
Video	120s	Same P50/P95 clarification required.

7.2 Availability — Open, not specified in either source

An uptime target and defined maintenance window are required. Expected behavior when an AI provider is down but Kelal's own infrastructure is up (a distinct and more likely state than total outage) must be defined.

7.3 Concurrency

Peak concurrent users to design for is unset. The text-model quality candidate's 20 requests/minute ceiling already bounds this to roughly 10 concurrent generations/minute regardless of target — the product team should state the target and the queuing behavior beyond it.

7.4 Localization & Accessibility

- Generated *content* is bilingual (EN/AM) — this is settled.
- **Open (OQ-14):** whether the *application interface itself* is localized into Amharic — a distinct question from bilingual content output, affecting every screen, so it must be answered before UI work begins.
- Minimum required regardless: respect for system font scaling, minimum contrast ratios, screen-reader labels on all interactive elements.

7.5 Device Support

No minimum OS version, RAM floor, or test-device matrix is specified in either source. Required, tied to the canvas-editor memory strategy in 6.9: a named minimum supported device, so the downscaled-proxy-image approach is tested against a real constraint rather than assumed adequate.

7.6 App Size

No budget specified. Required: a maximum installed-size budget, platform-appropriate split builds by architecture, and Ethiopic-preserving font subsetting per 6.7.

7.7 Mobile Data Cost

Not addressed in the Program Brief. Because mobile data cost is meaningful relative to income in the target market, and because repeatedly transferring full-resolution images is a cost the user directly bears: aggressive local caching of generated assets, never re-downloading an asset already held on-device, and consideration of a data-saver mode are required. A target for data consumed per generated post should be set (the client-side-rendering decision in 6.6 already works in this direction).

7.8 Security

- No provider API key may ever reach the mobile client; all generation calls proxy through the backend (a key shipped in an application binary is a key that has been published).
- Enforced encryption in transit everywhere; certificate pinning is an explicit yes/no decision (pinning complicates incident recovery — decide deliberately, not by default).
- Dependency vulnerability scanning runs in CI, not as a one-time Week 4 activity (the brief currently schedules the entire "focused security pass" for Week 4 — see below).
- Backup/recovery: "verified backups" is required but no recovery point or recovery time objective is set, and untested restores are not backups — a restore rehearsal with a recorded date is required before launch.

Sequencing note: the Program Brief allocates security to a single Week 4 "focused security pass." The structural items above (token handling, quota-as-financial-control, upload hardening, provider-key isolation) are foundational and should be designed in from Week 1; a Week 4 pass should verify these decisions were followed, not make them for the first time.

7.9 Data Protection, Prompt Confidentiality & Content Ownership — Open, all items

- **(OQ-13, blocking for public beta):** user prompts are effectively unannounced business plans (e.g. "50% off starting Friday" is competitively sensitive until published). Whether each selected AI provider's free tier logs or trains on submitted prompts is unverified and must be checked before any real user data is sent; if a provider requires consent to training, the product team must decide whether that's acceptable and, if so, disclose it plainly rather than via an unopened link.
- **(OQ-19):** applicable Ethiopian data-protection law, any data-residency requirement, and obligations arising from providers' own jurisdictions are unassessed — requires legal review, not engineering judgment, and should start early since a residency finding could change the hosting architecture.
- Independent of the legal position: a privacy policy, a defined retention period for generated content and prompts, and consent for telemetry collection are required regardless of jurisdiction.
- **(OQ-10):** whether provider terms permit commercial use of generated output, and whether providers claim any rights in it, is unverified for every selected provider — foundational, since the product's entire purpose is producing commercial material.
- **(OQ-11, covered in 6.5):** background-music licensing for video teasers.

7.10 Payment-Agnosticism — Recommended, adopted

In-app payment is confirmed out of scope for V1 (unchanged). The account/user data model should avoid baking in assumptions tied to a specific international payment provider's customer-identifier shape, since the dominant payment rails in the target market differ from those most integrations assume, and an entangled data model is awkward to unwind later. This costs nothing to do now.

8. User Flows

8.1 The Working Product Standard (end-to-end flow, unchanged from brief)

A business owner on a standard Ethiopian mobile network (3G/4G) opens Kelal Studio, enters a short business idea in Amharic or English, receives an accurate bilingual caption with relevant local hashtags, generates a branded graphic with correct Ethiopic text overlays, and exports it to Telegram or Instagram via the native OS Share Sheet — with zero technical assistance.

8.2 Core composer flow

1. **Sign up / log in** → session established.
2. **Idea entry:** free-text input, language auto-detected; platform target selected (Instagram/TikTok/Telegram).
3. **Generation:** bilingual caption + hashtags returned (target: 3s); if generation fails, a cached template is returned instead of a technical error.
4. **Graphic composition:** branded 1:1 (and 4:5, pending OQ-02) visual generated from the caption + Brand Kit (target: 6s).
5. **Canvas edit (optional):** user drags/resizes/edits 1–2 text layers on-device, safe-zone guides visible, Ethiopic spacing live.
6. **Export:** image saved to gallery, caption copied to clipboard, native Share Sheet opens with Instagram/TikTok/Telegram selectable.
7. **Manual paste:** user long-presses to paste the caption inside the target app (Kelal does not read or write the clipboard programmatically beyond the initial copy).

8.3 Draft continuity flow

User starts a draft → loses network or force-closes the app mid-edit → reopens the app → draft is intact exactly as left, sourced entirely from local device storage (no server round trip required, per 6.10 and its lifecycle diagram).

8.4 Video teaser flow

User submits a short storyboard → job enters a background queue (non-blocking) → system generates 3-part storyboard text, renders keyframe graphics per part, composites frames with text overlays and licensed background audio → user is notified on completion (target: 120s) → video available in the draft/export flow. (See the pipeline diagram in 6.5.)

8.5 Reminder flow

User schedules a Local Post Reminder on a draft → notification fires at the scheduled time (calendar/time convention per OQ-16) → tapping it opens the draft directly into the export flow (8.2, step 6 onward).

8.6 Admin flow

Admin logs into the web portal → views live usage counts, per-user limits, flagged-prompt audit log, and background-queue health → can adjust user limits and review flagged prompts, with all administrative actions themselves audit-logged (6.13).

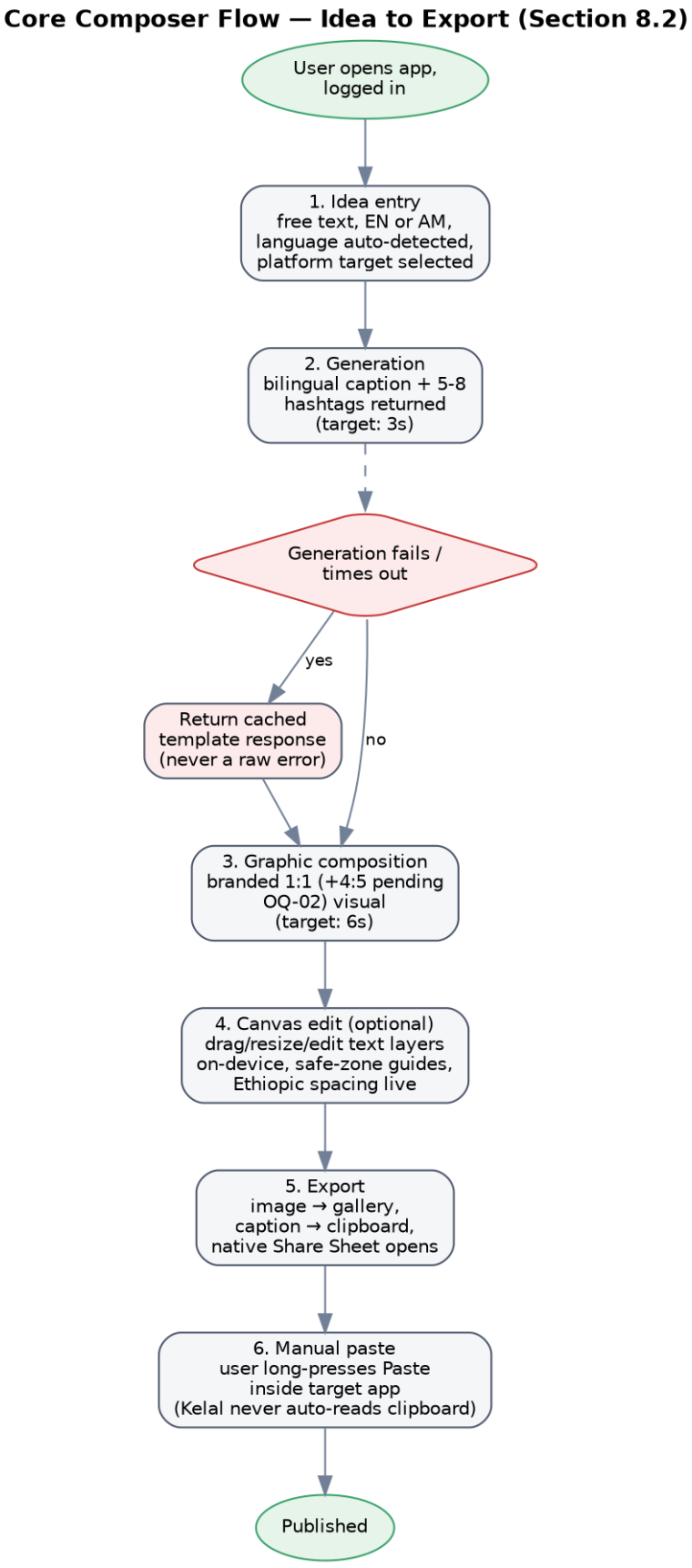

Figure 5: Core composer flow — idea to export, including the fallback path on generation failure

9. Acceptance Criteria

Mapped to the Program Brief's Delivery Checklist, each restated as a testable bar. All criteria are verified on a **physical device** under simulated 3G/4G conditions, per the Peer Test (a team member who did not build the feature completes the full flow unassisted).

#	Criterion	Acceptance bar
1	Self-registration	A new user can sign up and configure their account independently, end-to-end against staging, not just locally.
2	Account recovery	Password reset works reliably via email; the reset endpoint gives no signal about whether an email exists.
3	Account deletion	A user can delete their account in-app (added per 5.5).
4	Live AI generation	Given a raw business update in either language, the API returns a complete bilingual caption + hashtag payload within the agreed latency target on staging, with a working fallback on failure. Model selection has cleared the Amharic quality bar defined in 6.2 before this counts as done.
5	Ethiopic typography	Amharic text renders with correct spacing, zero missing glyphs, on every tested screen size and output surface — verified by the golden-image regression suite (6.7), not only a manual checklist.
6	Offline & low-network tolerance	A user can start a draft, lose network or force-close mid-edit, and reopen to find the draft exactly as left — verified with a deliberate airplane-mode test.
7	Clear user feedback	Failures (timeouts, moderation refusals, clipboard/gallery/share errors) trigger plain-language guidance, in the user's input language where applicable, never a raw technical error.

#	Criterion	Acceptance bar
8	Admin oversight	An admin can view live usage, error, and queue metrics, manage user limits, and review the flagged-prompt audit log; all admin actions are themselves logged.
9	Data safety & recovery	Backups run on schedule and a restore has been rehearsed and verified against staging data at least once before launch, with a recorded date.
10	Branded visual generation	Given a Brand Kit and a generated caption, the system produces a correctly branded graphic (ratio per OQ-02 resolution) within the agreed latency target, with no text or logo touching safe-zone boundaries (per platform, per OQ-06 resolution).
11	Canvas editing	A user can drag, resize, and edit text directly on the graphic on a physical device, safe-zone guides visible, Ethiopic spacing correct through every edit.
12	Export	Tapping export saves the image, copies the caption, and opens the Share Sheet with Instagram/TikTok/Telegram selectable, verified on both Android and iOS hardware; edge cases (app not installed, permission denied, write failure) show plain-language messages.
13	Video teasers	A submitted storyboard produces a complete 15–30s video with overlays and licensed audio within the agreed latency target, with completion notification, without blocking app usage.
14	Reminders	A scheduled reminder fires and opens the target draft directly into the export flow.

#	Criterion	Acceptance bar
15	Moderation	A disallowed prompt is blocked with a clear message (in the user’s input language) and logged for admin review, with a measured Amharic recall figure documented before public beta.
16	Quota enforcement	A user who exhausts their quota sees a clear message and a reset time before being able to attempt further generation; admins are alerted before the shared ceiling is reached.

10. Architecture Decisions

10.1 Provider abstraction with explicit failover. No feature code references an AI provider directly. A single internal interface (normalized request/response, per-provider timeout, typed error taxonomy distinguishing quota exhaustion / timeout / moderation refusal / malformed output, per-call usage telemetry) sits in front of a configurable, ordered failover chain. This is not speculative flexibility — free model rosters have been observed to change monthly and free tiers get withdrawn without notice, so at least one chosen provider becoming unavailable within a year is a likely event. This choke point is also what makes per-user quota enforcement (6.14), caching (10.2), and the Amharic evaluation (6.2) possible to implement at all. See the failover diagram in Section 6.14.

10.2 Caching and idempotency. Generation results are cached, keyed on a hash of (normalized input + brand identifier + model version). Users retry on slow networks and app backgrounding; every uncached retry burns scarce provider quota reproducing a result the system already had. Given quota is the binding capacity constraint (Section 12), this is a capacity multiplier, not a performance nicety. **Open:** cache TTL, whether the cache is shared across users (privacy implication), and whether a user can force regeneration.

10.3 Asynchronous work (video queue). Queue technology and job durability across service restarts, retry policy and backoff, behavior when a queued job's source draft/account is deleted mid-flight, max queue depth and user-facing behavior at that depth, job expiry, and notification channel (push vs. in-app, noting push has its own setup lead time) are all **required and currently unspecified** by either source document.

10.4 API contract before Week 1 ends. Three codebases (mobile, web, backend) are being built in parallel against a contract that does not yet exist — "Validated System API Specifications" is elevated from a final deliverable to a **Week 1** deliverable. The specification should be machine-readable (OpenAPI), with client types generated for both Flutter and web, so contract drift becomes a compile error at integration time rather than a runtime bug discovered during launch hardening. See Section 11 for the initial endpoint skeleton.

10.5 Data model.

Beyond the Brand Kit (the only entity the Program Brief actually describes), the following are modeled at minimum: **User** (with a deletion path, 5.5); **BrandKit** and its asset references; **Draft** (device-local identity and state in V1, per 6.10); **GenerationRecord** (input, output, model, provider, cost, latency — the entity the Week 4 telemetry requirements silently depend on; without it, "average rendering latency" and "export completion rate" cannot be reported); **Asset** (storage reference, dimensions); **Job** (async video work); **ModerationFlag** (audit log); **QuotaConsumption** (per-user, per-day); **AdminAuditLog** (admin action trail, 6.13); **RefreshToken** (rotation and reuse detection, 6.1).

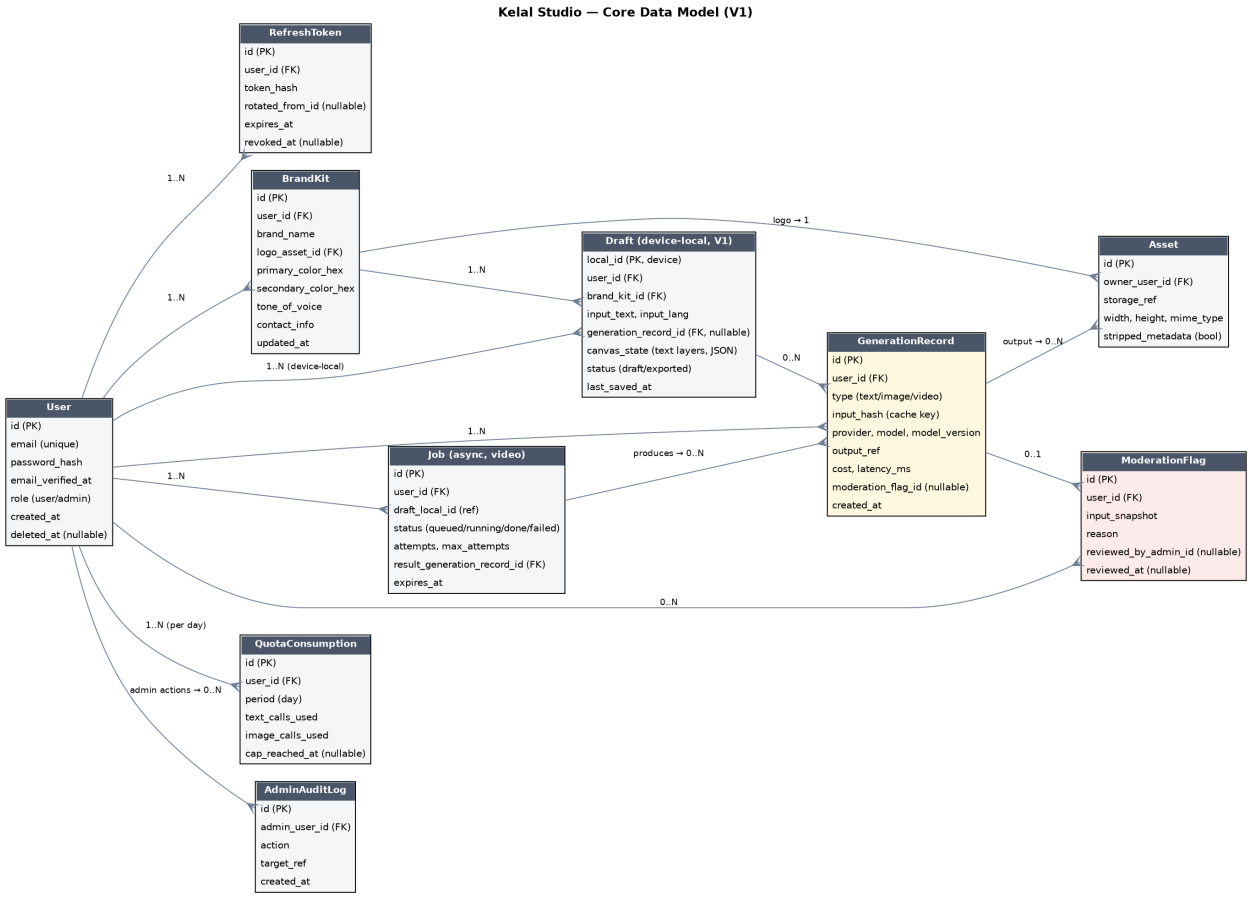

Figure 6: Core data model — entities, keys, and relationships for V1

11. API Contract Skeleton

This is a starting skeleton for the Week 1 OpenAPI specification (10.4), not the full spec. Request/response bodies are illustrative field lists, not final schemas — the API Owner should formalize this into OpenAPI YAML before parallel client work begins. All endpoints are served behind the API Gateway; all AI-provider calls happen server-side only, never from a client.

Method & Path	Purpose	Key request fields	Key response fields	Auth
`POST /v1/auth/register`	Create account	email, password	user_id, verification_sent	none
`POST /v1/auth/login`	Authenticate	email, password	access_token, refresh_token	none
`POST /v1/auth/refresh`	Rotate tokens	refresh_token	access_token, refresh_token	refresh token
`POST /v1/auth/password-reset/request`	Start reset	email	generic 200 always (no enumeration)	none
`POST /v1/auth/password-reset/confirm`	Complete reset	reset_token, new_password	200	none
`DELETE /v1/auth/account`	Delete account (5.5)	confirmation	202	access token
`GET/PUT /v1/brand-kits/{id}`	Read/update Brand Kit	logo_asset_id, colors, tone, contact	brand_kit object	access token
`POST /v1/assets`	Upload logo/media	file (multipart), declared_type	asset_id, storage_ref	access token
`POST /v1/generate/text`	Bilingual caption + hashtags	idea_text, lang_hint?, platform, brand_kit_id	caption_en, caption_am, cta, hashtags[], generation_id	access token, quota-gated (6.14)
`POST /v1/generate/image`	Branded graphic	generation_id (from text step) or caption payload, ratio, brand_kit_id	asset_id, url, generation_id	access token, quota-gated
`POST /v1/generate/video`	Queue a video teaser	storyboard_text or idea_text, brand_kit_id	job_id, status:queued	access token, quota-gated
`GET /v1/jobs/{id}`	Poll/receive job status	—	status, result_generation_id?	access token
`POST /v1/moderation/check`	Pre-generation filter (internal, called by generate/*)	text, lang	allowed:boolean, reason?	internal
`GET /v1/quota/me`	Remaining quota before attempting generation (6.14)	—	text_remaining, image_remaining, resets_at	access token

Method & Path	Purpose	Key request fields	Key response fields	Auth
`GET /v1/drafts` …	Draft sync — **only if OQ-05 resolves toward sync**; V1 default is device-local only, no server endpoint needed	—	—	—
`POST /v1/reminders`	Schedule a Local Post Reminder	draft_local_id, fire_at (UTC)	reminder_id	access token
`GET /v1/admin/usage`	Admin dashboard data	date range	counts, latencies, error_rate	admin token
`GET /v1/admin/flags`	Flagged prompt audit log	—	flag list	admin token
`POST /v1/admin/flags/{id}/review`	Review a flagged prompt	decision, note	200	admin token (logged, 6.13)
`PUT /v1/admin/users/{id}/limits`	Adjust a user’s quota	new_limits	200	admin token (logged, 6.13)

Typed error taxonomy (returned by all `/generate/*` endpoints, per 10.1): `quota_exceeded` (with `resets_at`), `provider_timeout`, `moderation_refused` (with `reason`, localized `message`), `malformed_output`, `validation_error`. Clients branch UI messaging on this field, not on HTTP status alone — this is what makes “helpful, plain-language guidance” (the brief’s own stated bar) implementable instead of a generic failure screen.

12. Capacity & Cost Constraints

These figures come from the verified provider pricing in Section 6.2 and directly determine the numbers required in 6.14 (quota exhaustion is an expected daily condition, not an edge case). Recheck before Week 1 build starts — vendor pricing and quotas change without notice.

- **Text (quality candidate, pre-$10-credit):** ~50 requests/day system-wide ≈ 25 caption generations/day across the *entire* user base (dev-sandbox scale only). Post one-time $10 credit: ~1,000 requests/day ≈ 500 generations/day, supporting roughly a 50-user beta at 10 generations each.
- **Concurrency ceiling:** 20 requests/minute ≈ 10 concurrent generations/minute — this, not the daily cap, is what a demo or launch moment with simultaneous users will actually hit.
- **Images (quality candidate, free tier):** at 1080×1080/1080×1350 output, roughly 61 posts/day (image-only) or 24 posts/day (with video, at 5 images/post) system-wide. Rendering at 1024×1024/1024×1280 instead of 1080 crosses one fewer 512px tile boundary and raises this to roughly 80 posts/day (image-only) — a ~30% capacity gain with an imperceptible quality cost after platform recompression. **Recommended** unless the product team identifies a reason 1080 output is required.
- **Unverified assumption underlying all image figures (OQ-12):** whether Cloudflare's per-step image cost is charged per image or per tile is not documented and must be measured on day one by generating a single image and reading actual neuron consumption. If charged per tile, every capacity figure above falls by roughly 4–5×. Do not plan capacity commitments on this arithmetic until measured.
- **Rate limits are enforced per account, globally — not per end user.** This is the direct justification for the in-app quota enforcement requirement in 6.14: a single active user can exhaust the whole beta population's shared budget in minutes, and the failure is invisible until the provider starts rejecting everyone.

13. Risks

Combined register — the Program Brief's five original risks plus items surfaced by the technical gap analysis that would not appear on a generic risk register.

Risk	Impact	Likelihood	Mitigation
Generation latency spikes	High	High	Strict timeouts, pre-cached fallback templates, provider failover chain (10.1).
Ethiopic text line collision	High	High	Line-height/padding constants (6.7) enforced via golden-image regression testing in CI, not manual checklist.
Video queue backlog	Medium	High	Cap at 720p/30s; frame-compositing (not generation) keeps per-post cost predictable.
Mobile network instability during demo	High	Medium	Local draft persistence; pre-rendered local backups on demo hardware; a pre-recorded fallback walkthrough for the Live Production Walkthrough, given the launch review is otherwise a single point of failure on live cellular data.
Initial beta onboarding friction	Medium	Medium	Seed onboarding through direct local business networks and creator channels.
Amharic generation quality is unproven	**Highest**	Unknown until measured	Blind native-speaker evaluation gates model selection in Week 1 (6.2) — the risk register's most conspicuous prior omission given it carries 20% of the evaluation weight.
Two renderers producing divergent output	High	High (by construction, if not resolved)	Single authoritative client-side renderer (6.6).

Risk	Impact	Likelihood	Mitigation
Provider free-tier withdrawal/repricing mid-project	Medium–High	Medium (observed pattern)	Provider abstraction with failover (10.1).
Shared quota exhausted by a single user	High	High under real usage	In-app per-user quota enforcement before outbound calls (6.14).
Region-wide connectivity loss (beyond intermittent drops)	Unassessed	Unassessed (OQ-17)	Local knowledge required; cheap mitigation (pre-recorded demo) regardless of answer.
Store rejection over missing account deletion	Medium	Plausible	Built into P0 regardless of OQ-18's resolution (5.5).
Unlicensed video background audio	Medium	Unknown	Resolve source/license before any public video output (OQ-11).
Prompt confidentiality with AI providers	Unassessed	Unassessed	Treated as blocking for beta pending OQ-13.

14. Open Questions Register

These are not decided. Nothing in this register may be treated as settled by anyone building against this PRD, including an AI coding tool asked to implement it. Each needs an owner and resolution date assigned by the product team.

ID	Question	Blocks	Priority
OQ-01	What did the Program Brief's missing sections 8, 9, 10 contain?	PRD authoring generally	Immediate — may resolve other items at zero cost
OQ-02	Two output aspect ratios (1:1, 4:5) or three (+9:16)? The Program Brief contradicts itself across four places.	Render pipeline, capacity model	Immediate
OQ-03	Is client-side rendering ratified as the single authoritative renderer (6.6)?	Mobile/backend architecture	Week 1
OQ-04	Confirm Ethiopic needs no complex text-shaping support in the chosen layout engine.	Typography implementation	Week 1 — cheap to confirm now, expensive to discover wrong later
OQ-05	Are drafts device-local only in V1 (as this PRD assumes), or synced?	Data model, sync work	Data model design
OQ-06	What are the actual safe-zone values, per platform and per ratio?	Canvas editor, render	~1 hour of validation work
OQ-07	What is the measured moderation recall on Amharic?	Launch readiness	Blocking for beta
OQ-08	What are Gemini's actual free-tier RPM/RPD for the selected model?	Capacity planning	Read from account dashboard once created
OQ-09	What is Pollinations' actual rate limit and behavior at limit?	Fallback design	Before relying on it as a fallback
OQ-10	Do provider terms permit commercial use of generated output, and do providers claim any rights in it?	Public beta, ToS	Foundational

ID	Question	Blocks	Priority
OQ-11	Source and license of background music for video teasers?	Video feature	Before any public video output
OQ-12	Is Cloudflare’s image step-cost charged per image or per tile?	Capacity planning	Day one — generate one image, read the dashboard
OQ-13	Do selected providers log or train on submitted prompts?	Public beta	Blocking for beta
OQ-14	Is the app interface itself localized into Amharic (separately from generated content)?	All UI work	Before UI work begins
OQ-15	Can target users actually type Amharic on their phones, or is transliteration the norm?	Composer design	User research, before composer design
OQ-16	Should dates/times reflect the Ethiopian calendar convention?	Reminders, scheduling	User research
OQ-17	What is the current region-wide connectivity risk over the delivery window (beyond intermittent network drops)?	Launch/demo planning	Requires local knowledge
OQ-18	Does app-store review require in-app account deletion? (This PRD builds it regardless — 5.5.)	Week 4 submission	Confirm against current store guidelines
OQ-19	What data-protection and residency obligations apply?	Hosting architecture	Legal review, start early
OQ-20	Which text model is primary, based on measured Amharic quality (not parameter count or vendor reputation)?	All generation work	Immediate — Week 1, ~2 days of evaluation work

15. Explicitly Out of Scope (P2 — reference)

Unchanged from the Program Brief: direct Meta/TikTok automated posting, a manual keyframe timeline video editor, multi-user team review/approval workflows, and in-app payment processing. None of these are reconsidered by this PRD.

This document synthesizes the Program Brief (scope, schedule, delivery standard) with the verified technical gap analysis (constraints, provider facts, risks, open questions), both reviewed 12 August 2026. Where the two sources agreed, content was merged without restating both. Where they conflicted or the gap analysis flagged an item as unverified, that status is preserved above rather than resolved by inference: an open item silently read as a settled answer is the most common way a project like this fails.